Übungsblatt 4: Gemeinsame Ressourcen, Deadlock und Prioritätsinversion

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung

Ziele und Ablauf

Sie erkennen verlorene Updates, wählen Schutzmechanismen und erklären Deadlock sowie Priority Inversion. Grundlage sind Vorlesungen 1–4, insbesondere Ready, Running und Blocked. CMSIS-RTOS2 bezeichnet die Echtzeitkernel-Schnittstelle des Common Microcontroller Software Interface Standard.

Verbindlicher Übungskern ist die Klausurvorbereitung auf Papier. Die angegebenen Punkte dienen zur Selbsteinschätzung; die Modulnote entsteht durch die Papierklausur. Alle Rechen- und Analyseaufgaben sind mit den angegebenen Daten ohne Hardware oder erfolgreiche frühere Programmieraufgaben lösbar. Begründen Sie Entscheidungen und nennen Sie Modellannahmen.

Abschnitt	Minuten	Zweck
Papierkern mit Besprechung	50	32 Orientierungspunkte
Transfer und Besprechung	20	Rechnen und begründen
Praxisfall: vorbereitete Demonstration oder Papieralternative	15	Vorhersagen, erklären, Grenzen nennen

Der Übungsblock umfasst **85 Minuten**. Die Praxisphase umfasst etwa fünf Minuten Vorhersage, höchstens fünf Minuten Demonstration und fünf Minuten Erklärung. Ohne funktionierenden Aufbau bearbeiten Sie in denselben 15 Minuten den vollständig vorgegebenen Papierfall. **Nach spätestens drei Minuten technischer Fehlersuche wird zum Papierfall gewechselt.** Dadurch bleibt der Übungsblock auch bei Setup-Problemen nutzbar.

Für eine Demonstration verwendet die Lehrperson **STM32CubeMX**, **VS Code mit STM32 VS Code Extension** und **NUCLEO-H755ZI-Q / STM32H755ZIT6**. Sie bereitet die geprüften Projektvarianten vor dem Termin vor. Installation, Projektgenerierung, Integration, Regenerierungsprüfung, Build und Flash sind keine Pflichtaufgaben im Übungstermin. Eine eigene Implementierung ist freiwillige Vertiefung. Alle Zahlen dieses Blatts sind Lehrannahmen, soweit sie nicht ausdrücklich als eigene Messwerte gekennzeichnet sind.

Der Übungsblock ergänzt 75 Minuten Erklärung und 20 Minuten Quiz/Abschluss zum Termin mit 180 Minuten. Die 50 Minuten Papierkern können wie bisher in 20, 15 und 15 Minuten zwischen Erklärblöcken verteilt werden.

Teil A: Papieraufgaben

Aufgabe 1: Verlorenes Update und konsistenter Snapshot (6 Punkte)

Maschinenmodell, kein nachgewiesenes Compilerlisting: Gemeinsamer Zähler startet bei 5. Jede Erhöhung besteht aus Laden, Addieren und Schreiben. A liest 5 und berechnet im eigenen Register 6. Danach wird A unterbrochen. B führt sein vollständiges Inkrement aus. Anschließend schreibt A seinen gesicherten Registerwert.

1. Ergänzen Sie Register A, Register B und gemeinsamen Speicher für alle Schritte. Welcher Endwert entsteht statt des gewünschten Wertes? (2 Punkte)
2. Reicht es, nur die beiden Stores einzeln zu schützen oder `volatile` hinzuzufügen? Begründen Sie den notwendigen Schutzumfang. (2 Punkte)
3. Bei einem Sensorpaar gehören `value` und `check` zum selben Update. Der Schreiber benutzt einen Mutex, der Leser nicht. Ist der gelesene Snapshot damit sicher konsistent? Beschreiben Sie eine korrekte Leseweise. (2 Punkte)

1. A liest 5, erhöht sein Register auf 6, Speicher bleibt 5. Beim Wechsel bleibt As 6 im gesicherten Kontext. B liest 5, erhöht sein Register auf 6 und schreibt 6. A schreibt anschließend seine 6. Der Speicher endet bei 6 statt 7.
2. Zwei einzeln geschützte Stores verhindern nicht, dass vorher beide denselben alten Wert lesen. Der gesamte zusammengehörige Lese-/Änder-/Schreibzugriff muss gegenüber dem anderen Teilnehmer unteilbar organisiert werden. `volatile` bietet diese Synchronisation nicht. Das gezeigte Maschinenmodell ist keine Empfehlung für tatsächlichen C-Mehrthreadcode mit konkurrierenden nichtatomaren Zugriffen: Ein Data Race kann dort undefiniertes Verhalten verursachen.
3. Der Leser kann zwischen den Schreiberzuweisungen landen und neuen `value` mit altem `check` erhalten. Er muss denselben Mutex erfolgreich erwerben, beide Felder in eine private Kopie übernehmen und freigeben. Längere private Berechnung folgt danach.

Aufgabe 2: Maskierungsbudget und Tokenzahl (8 Punkte)

1. Eine betroffene Interruptanforderung entsteht bei noch 8 µs verbleibender Maskierung. Der Eintritt benötigt danach angenommene 2 µs, die Reaktion weitere 3 µs. Weitere Kosten fehlen im Modell. Gilt $R \leq D$ für $D = 10\,\mu s$? Welcher maximale Maskierungsrest wäre unter denselben Annahmen zulässig? (3 Punkte)
2. Ein Semaphor wird mit Maximum 3 und Anfangszahl 3 angelegt. A, B und C erwerben je ein Token. D versucht Acquire mit Timeout 0. Danach gibt B frei, ohne dass ein anderer Thread wartet. D versucht erneut. Geben Sie die Zählerfolge und Ds beide Ergebnisse an. Ist mit dem Zähler die freie Pufferadresse bekannt? (3 Punkte)
3. Ein binärer Ereignissemaphor startet mit 0. Zwei Releases erfolgen, bevor jemand ein Token verbraucht. Wie viele Token sind anschließend verfügbar? Warum passt für „Ereignis noch nicht eingetreten“ der Startwert 0, für „Ressource anfangs verfügbar“ dagegen 1? (2 Punkte)

1. $R = (8 + 2 + 3)\,\mu s = 13\,\mu s > 10\,\mu s$. Die Modellfrist wird um 3 µs verfehlt. Der zulässige Maskierungsrest ist höchstens $(10 - 2 - 3)\,\mu s = 5\,\mu s$. Der Maskierungsumfang muss die tatsächlich konkurrierenden Interrupts umfassen; eine Threadmaskierung hält beispielsweise DMA nicht an.

2. Zählerfolge $3 \to 2 \to 1 \to 0$. Ds erster Versuch liefert einen Fehler ohne Blockieren. Bs Release erhöht auf 1; Ds **neuer** Versuch kann erwerben und reduziert auf 0. D war nach dem ersten Fehler kein registrierter Warter. Der Zähler gibt nur die Anzahl, nicht die Identität eines freien Puffers an.
3. Ein Token ist verfügbar. Das erste Release gelingt, das zweite scheitert am Maximum 1. Null lässt den Ereignis-Worker zunächst warten; eins stellt sofort eine Ressourcenerlaubnis bereit. Der Anfangswert beschreibt Bedeutung, nicht nur eine beliebige Konfiguration.

Aufgabe 3: Deadlock, Ordnung und Timeout (6 Punkte)

T1 hält Mutex A und wartet mit unbegrenzter Warteoption auf B. T2 hält B und wartet ebenso auf A. Es gibt genau eine exklusive Instanz je Ressource und keinen weiteren Abbruchpfad.

1. Zeichnen Sie den Ressourcenwartegraphen. Beschriften Sie die Bedeutung beider Pfeilrichtungen. Warum kann die betrachtete Gruppe nicht fortfahren? Müssen alle übrigen Threads und Interrupts stillstehen? (2 Punkte)
2. Erklären Sie die Coffman-Bedingung „No preemption“. Warum widerspricht sie einem präemptiven Prozessor-Scheduler nicht? Wie verhindert „immer A vor B“ den angegebenen Zyklus? (2 Punkte)
3. T1 bekommt beim Erwerb von B einen endlichen Timeout, hält aber schon A. Was muss sein Fehlerpfad tun? Warum garantieren Timeout und sofortiger Neuversuch allein keinen Fortschritt? (2 Punkte)

1. Mutex A $\to$ T1 $\to$ Mutex B $\to$ T2 $\to$ Mutex A. Ressource zum Thread bedeutet Besitz; Thread zur Ressource bedeutet Warten. Beide benötigen die vom jeweils anderen gehaltene Freigabe und bleiben Blocked. Andere Threads und Interrupts können trotzdem arbeiten.
2. „No preemption“ meint keinen zwangsweisen **Ressourcenentzug**, nicht fehlende Prozessorpräemption. Mit Ordnung A vor B darf T2 B nicht bereits halten, während es zuerst A fordert. Es wartet auf A ohne B; T1 kann B nehmen und beide freigeben.
3. Nach fehlgeschlagenem B-Erwerb nicht auf B zugreifen, gehaltenes A passend freigeben und teilweise Datenänderungen konsistent behandeln. Eine Wiederholungsregel muss festgelegt sein. Symmetrische sofortige Wiederholung kann fortgesetzte Konflikte ohne Nutzfortschritt erzeugen, einen Livelock. Prioritätsvererbung beseitigt den Ressourcenzyklus nicht.

Aufgabe 4: Derselbe Inversionsfall ohne und mit Vererbung (12 Punkte)

Ein Kern, feste Prioritäten $H > M > L$, einmalige Jobs, Wechsel- und API-Kosten null. API bedeutet Programmierschnittstelle.

Job	Freigabe	Prozessorbedarf
L	0 ms	nimmt Lock und benötigt insgesamt 5 ms bis zur Freigabe
H	1 ms	fordert denselben Lock, danach 1 ms eigene Arbeit
M	2 ms	20 ms Arbeit, greift nicht auf den Lock zu

Die relative Deadline von H ist $D_H = 10\,\text{ms}$. Der Anforderungsversuch von H hat im Modell keine eigene Dauer.

1. Zeichnen Sie die CPU-Belegung bis 26 ms **ohne** Vererbung. Bestimmen Sie Ls verbleibende Arbeit bei 1 und bei 2 ms. (3 Punkte)

2. Berechnen Sie Hs Lockwartezeit, Antwortzeit R_H und absolute Deadline. Ist die Frist eingehalten? (3 Punkte)
3. Zeichnen Sie dieselben Jobs **mit einem vererbenden Mutex**. Wann beginnt und endet Ls Anhebung? Bestimmen Sie Hs Warte- und Antwortzeit. (3 Punkte)
4. Vergleichen Sie die gesamte Prozessorarbeit. Welche Verzögerung wurde entfernt, welche nicht? Warum ist das kein Deadlock, und warum ist die endliche M-Arbeit von dauerhaftem Ready-Verhalten zu unterscheiden? (3 Punkte)

Ohne Vererbung:

Intervall in ms	CPU
$[0, 2)$	L, Hs dauerloser erfolgloser Versuch bei 1
$[2, 22)$	M
$[22, 25)$	L
$[25, 26)$	H

Bei 1 ms bleiben vier, bei 2 ms drei Millisekunden L-Arbeit. H wartet $25 - 1 = 24\,\text{ms}$ und hat $R_H = 26 - 1 = 25\,\text{ms}$. Seine absolute Deadline ist $d_H = 1 + 10 = 11\,\text{ms}$. Fertigstellung 26 ms liegt 15 ms danach.

Mit Vererbung: L arbeitet von 0 bis 5. Von 1 bis 5 hat es die hohe effektive Priorität, weil H am gehaltenen Mutex wartet. H läuft von 5 bis 6, M von 6 bis 26. H wartet $5 - 1 = 4\,\text{ms}$ und hat $R_H = 6 - 1 = 5\,\text{ms} \leq D_H$. Die Modellfrist wird gehalten.

In beiden Fällen beträgt die Gesamtarbeit $5 + 1 + 20 = 26\,\text{ms}$. Entfernt wurde M als zusätzliche Verdrängung des L-Halters vor H. Ls vier Millisekunden Restarbeit bleiben. L könnte bei Zuteilung fortfahren; es gibt keinen geschlossenen Ressourcenzyklus. Ein einmaliger M-Job endet nach 20 ms, ein dauerhaft bereites M könnte ohne Vererbung L unbegrenzt ausschließen. Bei mehreren Mutexen oder Gerätewartezeiten gilt dieses einfache Ein-Mutex-Ergebnis nicht pauschal.

Transfer: Nachrichtenschutz und asynchroner Abschluss · 20 Minuten

Bearbeiten Sie zehn Minuten und besprechen Sie zehn Minuten. Keine zusätzlichen Punkte. IT bedeutet interruptgesteuerte Übertragung, DMA Direct Memory Access, direkten Hardware-Speicherzugriff.

Ein UART sendet 100 Nutzbytes bei 115200 Baud im Format 8N1. UART ist die asynchrone serielle Schnittstelle, 8N1 bedeutet acht Datenbits, kein Paritätsbit, ein Stoppbit; ein Startbit kommt hinzu.

1. Berechnen Sie die minimale reine Leitungszeit unter diesen Annahmen.
2. Zwei Threads sollen ganze Zeilen ausgeben. Warum werden privater Puffer und gemeinsamer UART-Mutex benötigt, und weshalb ist `printf` nicht automatisch durch diesen Mutex insgesamt threadsicher?
3. Bei IT-/DMA-Übertragung kehrt die Startfunktion vor dem tatsächlichen Abschluss zurück. Wann darf der Puffer wiederverwendet werden und wer gibt den CMSIS-Mutex frei? Macht Prioritätsvererbung die Leitung schneller?

$100 \cdot 10/115200\,\mathrm{s} \approx 8{,}68\,\mathrm{ms}$ reine Leitungszeit. Privater Puffer erhält die eigene Nachricht, der gemeinsame Vertrag serialisiert das Gerät. Bibliothekszustände und tatsächliches Retargeting der formatierten Ausgabe sind separat zu prüfen. Ein sichtbares Mischbild wird nicht als universelles Soll behauptet.

IT ist die interruptgesteuerte Übertragung, DMA direkter Hardware-Speicherzugriff. Der Puffer bleibt bis zu bestätigtem Abschluss oder sicherem Abbruch gültig und unverändert. Der Callback darf zulässig signalisieren; der Besitzerthread gibt den Mutex frei. Ein Timeout allein bestätigt noch keine ruhende Hardware. Vererbung verändert die Taskzuteilung, nicht die Leitungsrate.

Teil B: Praxisfall · 15 Minuten

Praxisfall 4: Ablauf und Marker lesen

Papieralternative, synthetische Reihenfolgen: Drei Threads haben $H > M > L$ und verwenden genau eine Ressource. L hält sie vor Hs Anforderung. H blockiert am belegten Schutzobjekt; M ist dann Ready und greift nicht auf die Ressource zu. L hält keine weiteren Locks und wartet nicht auf Geräte.

Ohne Vererbung	Mit Vererbung
L erwirbt, H fordert, M arbeitet, L gibt frei, H erwirbt und endet	L erwirbt, H fordert, L gibt frei, H erwirbt und endet, M arbeitet

1. Erklären Sie die Reihenfolge und die verbleibende H-Warteursache. Welche Eigenschaft des Mutex ist erforderlich?
2. In einem zusätzlichen **synthetischen 1-kHz-Ticklog** liegen H-Anfrage und H-Erwerb beide bei 100, H-Ende bei 101. Beweist dies eine Lockwartezeit von exakt null? Ist der Unterschied zwischen Anfrage und Ende eigene CPU-Zeit?
3. L schreibt seinen Ende-Marker erst nach Rückkehr aus dem Release-Aufruf. Warum ist dieser Marker keine verlässliche genaue Ressourcenfreigabezeit?

Bei vorbereitetem Board: Die Lehrperson zeigt je einen fertigen Semaphor- und Mutexlauf des `lab04_resources`-Moduls. Die Gruppe erklärt die Auswahlreihenfolge. Neuaufbau und Laufzeitmessung mit zusätzlichen GPIOs sind freiwillig.

Musterlösung Praxisfall 4

Ohne Vererbung verdrängt M den niedrigen Halter L, während H auf Ls Freigabe wartet. Mit Prioritätsvererbung erhält L vorübergehend Hs effektive Priorität und kann vor M freigeben. H wartet weiter auf Ls verbleibende geschützte Arbeit; diese verschwindet nicht.

Gleiche Tickwerte geben keine Auflösung unter einem Tick und beweisen keine physische Nullwartezeit. Anfrage bis Ende enthält Warten und Arbeit, gegebenenfalls weitere Kosten, keine isolierte H-CPU-Zeit. Nach dem Release kann H oder M vor Ls Rückkehr laufen. Ls späterer Ende-Marker liegt dann nach der tatsächlichen Freigabe. Die Tabelle belegt nur das genannte einfache Modell, keine allgemeine Deadlinegarantie.

Ergebnis und freiwillige Vertiefung

Halten Sie die begründeten Papierlösungen (32 Orientierungspunkte), Ihre Transferantworten und die Erklärung des Praxisfalls fest. Projektdateien, Versionslisten, Flashnachweise und Messprotokolle sind keine Pflichtabgabe dieses Übungsblocks. Für das Verständnis des nächsten Blatts ist keine erfolgreiche Firmwareintegration erforderlich.

Die freiwillige Programmiervertiefung enthält die bisherige technische Vorbereitung und Integration. Sie richtet sich an die Lehrperson sowie an Studierende, die außerhalb des Termins mehr Praxis wünschen. Ein fehlender Boardlauf beeinträchtigt die Bearbeitbarkeit der Papieraufgaben nicht. In eigenen Protokollen bleiben ungemessene Zeitfelder leer; beobachtete Stichprobenmaxima sind keine allgemeine Worst-Case-Garantie.

Quellen: Vorlesung 4, Arm CMSIS-RTOS2 zu Mutex und Semaphore, die **lokale** `cmsis_os2.c` und FreeRTOS-Konfiguration, ST UM2408/RM0399/AN5361 und Dokumentation der eingesetzten STM32 VS Code Extension. Der öffentliche Adapterstand ist keine automatisch bestätigte Firmwareversion Ihres Boards.